Three Axes of Quantum Risk: Why "PQ-Ready" Is Not Enough

Unified Observability for Post-Quantum Cryptography, Quantum Key Distribution, and Crypto-Agility

Aleaddin Özer (CIO, E2E Solutions)
Murat Aydos (Assoc. Prof., Hacettepe University)

2026-05-13

ABSTRACT

Abstract
Operators preparing for the post-quantum transition face three independent questions that today's discovery tooling answers separately, if at all. Which primitives are quantum-resistant, which channels are additionally protected by quantum-secure key delivery, and how agile is each asset under migration pressure? We argue that quantum-risk posture is a three-axis problem, not one. We define algorithmic resistance (A), channel protection (C), and migration agility (G) as independent observables, and combine them with the operator's deadline horizon into a single quantum-risk score $q(\text{asset}, t)$ whose agility weight shrinks as the deadline approaches. We extend an open event schema with the new axes and release a reference implementation, Sezar: eBPF-based wire observation, an ETSI GS QKD 014 collector and reusable KME emulator, and a static crypto-agility scanner. Across three reproducible studies we measure 57% PQ-KEM adoption in a 30-host sample of well-known Web properties, validate channel-state classification on every induced transition (13/13) in a KME emulator, and reach 91% agreement with hand-graded ground truth on an 11-project agility corpus. Practitioners can reproduce our evaluation with no QKD hardware, no commercial scanner, and a single Linux host.

1. PQ-Ready Is the Wrong Question

In 2026 a network operator can read NIST's FIPS 203, 204, and 205 [1], [2], [3], scan their environment with one of several PQ-discovery tools [4], [5], [6], and learn how much of their TLS, SSH, and certificate inventory is "PQ-ready."

That answer is not wrong. It is incomplete in a way that matters operationally. Consider two TLS terminators sitting side by side in the same data center:

- **Terminator A** runs nginx 1.27 with a TLS configuration that negotiates `X25519` today but accepts `X25519MLKEM768` the moment the operator changes one configuration line.
- **Terminator B** is a vendor appliance whose cryptographic module is FIPS 140-3 validated against a tested configuration that lists only classical algorithms. To swap algorithms, the operator must wait for the vendor to issue a new firmware build, complete a new FIPS validation, and roll the device in a hardware-refresh cycle that historically runs 18 to 36 months.

Today's discovery tools assign both terminators the same label: *classical, not PQ-ready*. NSA CNSA 2.0 [7] gives both the same deadline. From any meaningful operational standpoint their *real* exposure could not be more different:

Terminator A is one configuration line away from compliance, while Terminator B requires a multi-quarter capital program. A dashboard that reports them identically is operationally misleading.

A parallel mismatch arises with Quantum Key Distribution (QKD) [8], [9]. A growing minority of high-assurance links — financial inter-datacenter fiber, government inter-site, metropolitan testbeds in EuroQCI member states [10], [11] — protect their session key material with QKD- delivered pre-shared keys (PSK) layered onto otherwise classical TLS or IPsec. The cryptographic primitives observed on the wire are unchanged; the *channel* through which session keys reach the endpoint is different. Discovery tools that look only at ciphersuites cannot see the difference.

Standards-body guidance [12], [13] is skeptical of QKD as a substitute for PQC at present — and that position is reasonable when the question is "where should I invest." From an *observability* standpoint the dichotomy elides a distinction operators need: PQC and QKD are complementary signals on the *same* asset. An asset using ECDSA-P256 over a QKD-protected link is in a different posture than an asset using ECDSA-P256 over an unprotected link, even though their algorithms are identical.

1.1 What Existing Tooling Does, and Does Not Do

Existing PQ-discovery work is mature on one axis. Cisco's PQC Discovery service combines passive network observation with active endpoint scanning and reports algorithm-class readiness across an environment [4]. IBM's Quantum Safe Discover [5] and parallel internal tooling at large operators [14] follow the same pattern. Cloudflare's measurement reports [6] provide one of the few public corpora on PQ TLS adoption. Academic measurement work has characterized TLS deployment hygiene for over a decade [15], [16], [17], and the NIST migration playbook [18] together with peer guidance from BSI [19], ANSSI [20], and the UK NCSC [21] provide the normative reference. QKD telemetry has been studied primarily from the QKD operator's perspective via the ETSI ISG-QKD documents [9], [22], not the consuming application's. Crypto-agility has been a recurring talking point since RFC 7696 [23] and is repeatedly named in PQ-migration documents as a precondition — yet, to our knowledge, no widely deployed inventory tool surfaces it as a first-class telemetry field. The result is a three-stranded literature feeding single-axis dashboards.

This article argues that quantum-risk observability requires three axes, not one:

- **A** — Algorithmic resistance (the primitive itself).
- **C** — Channel protection (the key-delivery mechanism).
- **G** — Migration agility (the cost of changing A).

In the rest of this article we define each axis, combine them into a single deadline-adjusted risk score, walk through a worked example, and describe an open reference implementation that any practitioner can deploy.

---

2. The Three Axes

2.1 Axis A — Algorithmic Resistance

Axis *A* scores the primitives an asset uses on [0, 1], where 1 corresponds to quantum-resistance under the standard NIST assumptions and 0 corresponds to a deprecated or broken algorithm. This is the axis today's tooling already addresses, and we adopt the classification consensus of NIST IR 8547 [18], BSI [19], and ANSSI [20]: ML-KEM-* and ML-DSA-* / SLH-DSA-* primitives score 1.0, hybrid constructions (e.g., `X25519+ML-KEM-768`) score 0.9, classical primitives at standard parameter sizes score 0.3, deprecated primitives (SHA-1, RSA-1024, MD5, RC4, 3DES) score 0.0, and unknown primitives default to 0.4 with an `unknown_primitive` flag, biasing toward operator review rather than silent acceptance.

A typical TLS 1.3 session contributes four primitive observations: key exchange, signature, AEAD, hash. We weight these by relevance to the harvest-now/decrypt-later threat —

signature 0.40, KEM 0.30, AEAD 0.20, hash 0.10 — and re-normalize when a subset is present. The resulting per-asset score is a recognizable algorithm-readiness number that operators can compare against existing dashboards.

2.2 Axis C — Channel Protection

Axis C scores the channel through which key material reaches the endpoint. Three categorical states cover the realistic deployment options:

- **classical** ($c = 0.0$): session key derived solely from negotiated cryptography.
- **qkd_hybrid_psk** ($c = 0.7$): session key derived from a QKD-delivered PSK combined (XOR or HKDF) with the negotiated KEM — the pattern documented in NIST SP 1800-38 [24] and emerging in TLS PSK drafts [25].
- **qkd_only** ($c = 1.0$): session key derived entirely from QKD material — rare, typically MACsec-class transport.

ETSI's *GS QKD 014* specification [9] gives us the stable observation surface. The Key Management Entity (KME) exposes `/status`, `/enc_keys`, and `/dec_keys` REST endpoints that any Secure Application Entity (SAE) — strongSwan, Wireguard, or a TLS terminator — calls to retrieve fresh key material. By polling `/status` we observe the link's quantum bit error rate (QBER), key rate, and health. By cooperating with the SAE we observe per-session attribution: *which* keys were consumed on *which* sessions.

The SAE may *think* it is operating in hybrid PSK mode while the underlying KME has degraded into a classical fallback. The role of an observability layer is to surface this discrepancy. We define c on observed KME state, not on SAE-reported intent.

2.3 Axis G — Migration Agility

Axis G scores the asset's ability to change its primitives. Where the crypto-agility literature [23] has discussed this property qualitatively, we operationalise it as five ordinal levels:

Level	g	Observable signature
`negotiated`	1.0	Algorithm selected per-session by protocol negotiation.
`configurable`	0.75	Algorithm fixed per-deployment; changeable by configuration alone.
`pinned`	0.50	Algorithm fixed in code; changeable by software upgrade.
`locked`	0.20	Algorithm fixed in firmware or by FIPS validation scope.
`frozen`	0.0	Algorithm fixed in silicon, ROM, or otherwise unchangeable without hardware replacement.

The classification derives from static analysis of the asset's implementation surface — configuration files, source code, binary strings, vendor declarations of FIPS scope. Returning to our two terminators from §1: Terminator A scores $g = 0.75$ (configurable nginx); Terminator B scores $g = 0.20$ (FIPS-locked vendor appliance). The single bit "classical / not PQ-ready" collapses these into one bucket. The agility axis restores the distinction operators actually need.

---

3. Combining the Axes

The three axes are independent observables. To collapse them into a single posture metric we adopt the operator's deadline D as a fourth input.

Let t be the current date. Define *deadline tension* $\tau(t) = \max(0, \min(1, 1 - (D - t)/H))$ for a horizon constant H (we default to five years). When D is far away, $\tau \to 0$; as $t \to D$, $\tau \to 1$.

We define the quantum-risk score

$$q(\text{asset}, t) = 1 - (\alpha \cdot A + \beta \cdot C + \gamma(\tau) \cdot G)$$

with default weights $\alpha = 0.5$, $\beta = 0.2$, $\gamma(\tau) = 0.3 \cdot (1 - \tau)$, re-normalized so the three weights sum to one as γ shrinks. The *agility* weight is the only weight that shrinks with

deadline tension. The intuition: agility is an operator's optionality, and optionality is worth less as the clock runs out. The same asset gets evaluated as "fixable" early and as "stuck with whatever you've got" late.

The score is intentionally a *prioritization* signal — it answers "where should I spend the next quarter's migration budget?" rather than "what is the absolute risk of this asset?" The dashboard pairs q with an orthogonal **BLOCKED** flag for assets whose $G \le 0.20$: such assets cannot be migrated by configuration or software update alone and require a vendor or hardware program regardless of how the priority score evolves. We discuss the interplay between q and `BLOCKED` in the worked example that follows.

Asset-class weights w_k further weight an asset's contribution to the org-wide posture: `blockchain_key` is weighted higher than `tls_session` (a forged signature against a public-chain key is permanent; a forged session is ephemeral), and `x509_cert` is weighted higher than `ssh_session` (a weak CA chain is a public liability that propagates downstream).

3.1 A Worked Example

To make the formula concrete we evaluate four representative assets at two time points: the present day (t_1 = 2026-05-13) and a date eighteen months before the NSA CNSA 2.0 browser / server class deadline (t_2 = 2029-07-01). All four assets share the same target deadline D= 2030-01-01 and horizon H = 5 years. At t_1 the deadline tension is $\tau_1 = 0.27$; at t_2 it is $\tau_2 = 0.90$. Renormalizing the weights so $\alpha + \beta + \gamma$ sums to one (per §3), at t_1 this gives $\alpha' = 0.544$, $\beta' = 0.218$, $\gamma' = 0.238$; at t_2, $\alpha' = 0.685$, $\beta' = 0.274$, $\gamma' = 0.041$.

The four assets:

- **α — Modern, agile, no QKD.** An nginx 1.27 terminator with TLS 1.3, `X25519 + ECDSA-P256 + AES-256-GCM + SHA-384`. Cipher list lives in `nginx.conf`. Configurable. $A = 0.51$, $C = 0.0$, $G = 0.75$.
- **β — Modern, FIPS-locked, no QKD.** Same observed primitives as α, but inside a vendor appliance whose FIPS 140-3 validated configuration enumerates only classical algorithms. $A = 0.51$, $C = 0.0$, $G = 0.20$. `BLOCKED` flag raised.
- **γ — Legacy, pinned, no QKD.** A Postfix SMTP server on an old distribution with TLS 1.2, `DH-2048 + RSA-2048 + AES-128-CBC + SHA-1`. Cipher list pinned in the version shipped by the vendor; agility requires package upgrade. $A = 0.12$, $C = 0.0$, $G = 0.50$.
- **δ — Modern, agile, behind QKD.** Same nginx as α but the session terminates inside a QKD-protected MACsec segment with hybrid PSK enabled; ETSI 014 status reports a healthy link. $A = 0.51$, $C = 0.7$, $G = 0.75$.

Table 1 reports the computed q for each asset at the two time points.

Table 1. Worked example of the deadline-adjusted quantum-risk score q at t_1 (2026-05-13, $\tau = 0.27$) and t_2 (2029-07-01, $\tau = 0.90$), with default weights and deadline D = 2030-01-01.

Asset	A	C	G	$q(t_1)$	$q(t_2)$	Trend	Flag
α modern-agile	0.51	0.00	0.75	**0.54**	**0.62**	rising	—
β modern-locked	0.51	0.00	0.20	**0.67**	**0.64**	flat	`BLOCKED`
γ legacy-pinned	0.12	0.00	0.50	**0.82**	**0.90**	rising	—
δ modern-QKD	0.51	0.70	0.75	**0.39**	**0.43**	rising	—

Three observations follow from the table. First, the legacy SMTP server (γ) dominates the priority list throughout — both its algorithmic content and its limited agility leave it as the worst-graded asset, and it remains so as the deadline approaches. Second, the modern but locked appliance (β) sits above the modern but

agile server (α) early in the window because its low agility is heavily penalized when the operator still has runway to act on it. As $\tau \to 1$ the agility weight shrinks and q_β slightly declines while q_α rises — the priority queue tightens. The dashboard keeps β visible regardless via the `BLOCKED` flag, which marks it as requiring a vendor or hardware program independently of the priority signal. Third, asset δ — algorithmically identical to α but protected by a QKD-PSK channel — is the lowest-priority asset in the example throughout. QKD's contribution partially compensates for classical primitives, which matches the deployment rationale for QKD on high-assurance links.

The same observables produce different scores at different t because the score is *prioritization-adjusted*, not asset-intrinsic. An operator using the score has a clear quarterly action list: focus on γ and `BLOCKED`-flagged β first, then α as its grace period erodes, and treat δ as a deferred maintenance item rather than a migration target.

Producing those numbers operationally requires a pipeline that observes all three axes uniformly. The rest of the paper describes Sezar, which provides exactly that, and the empirical work the pipeline supports.

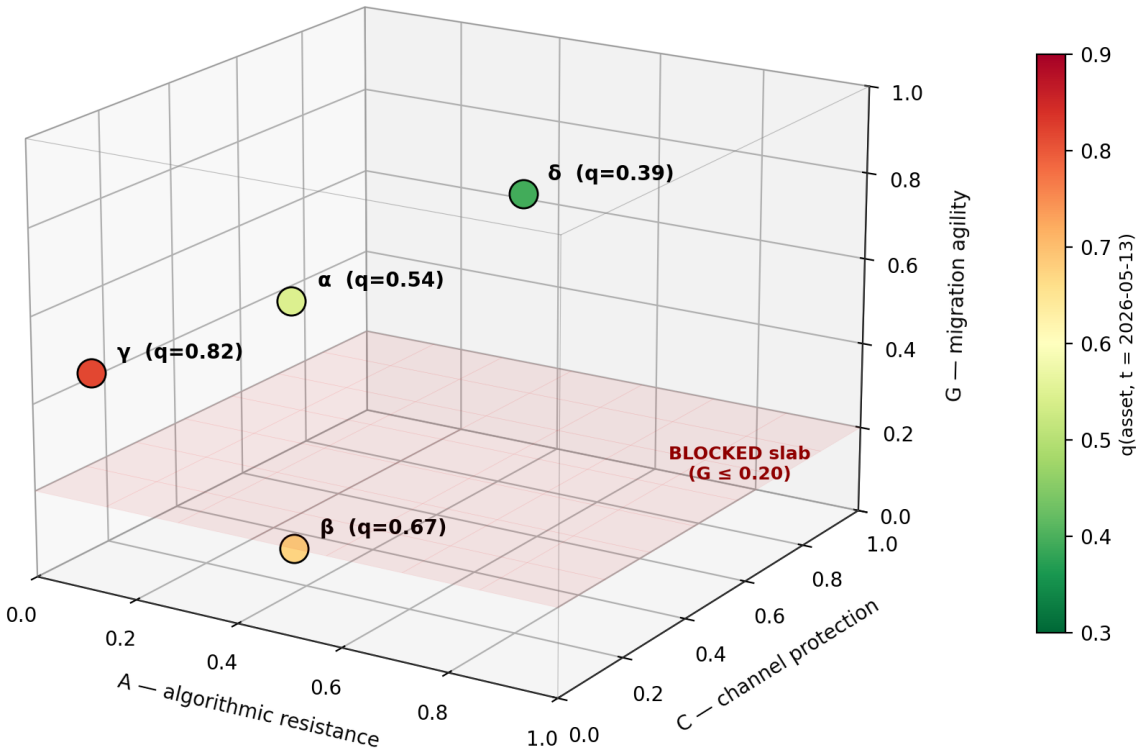

Figure 1. *The three-axis quantum-risk space. Each cryptographic asset occupies a point in (A, C, G); colour encodes the deadline-adjusted score q. The four worked-example assets (α, β, γ, δ) are plotted at their observed coordinates. The shaded region near the A = 0, G = 0 corner is the* `BLOCKED` *-flagged volume requiring out-of-band remediation.*

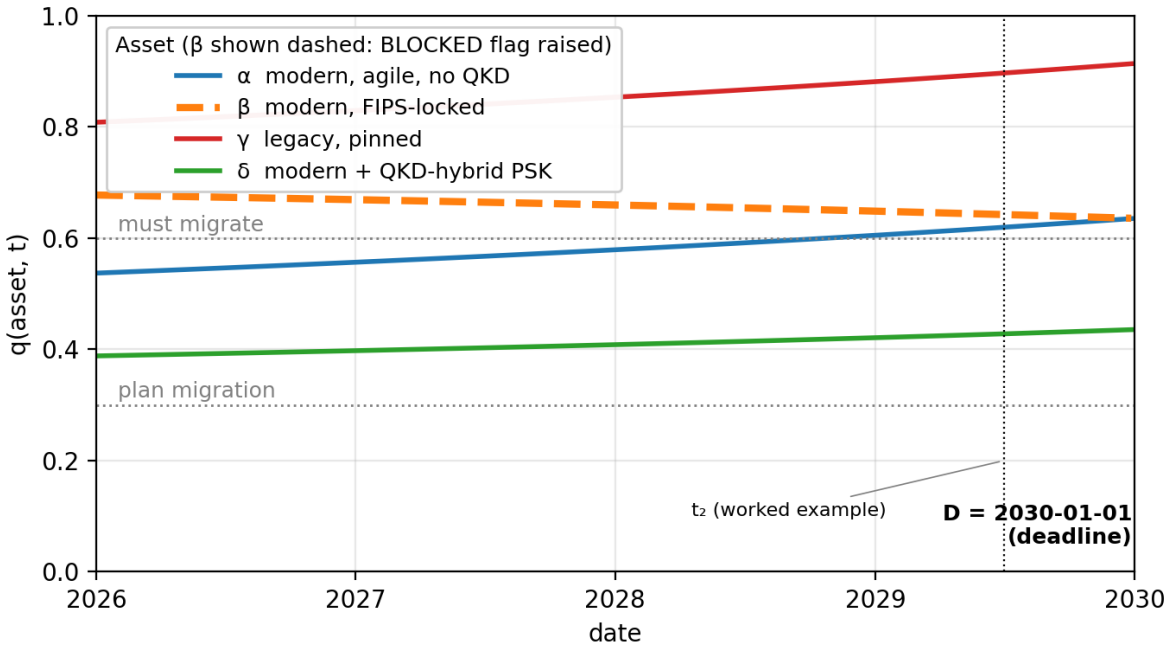

Figure 2. *Trajectory of q(t) for the four worked-example assets from 2026 to the deadline at 2030-01-01, holding observables fixed. The legacy-pinned asset (γ) climbs steepest; the modern-agile asset (α) rises across the q > 0.6 must-migrate threshold as its agility weight erodes. The locked-but-modern asset (β) remains high throughout — the* `BLOCKED` *flag, not the prioritization score, is what surfaces it for action.*

4. Sezar: A Reference Implementation

We have implemented the three-axis model in Sezar, an open-source Rust workspace released under MIT. The schema specification (`docs/crypto-event-schema.md`) and the posture-rollup formula (`docs/posture-rollup.md`) live next to the code; the rest of this section sketches the parts that matter for the magazine reader.

4.1 One Schema, Many Agents

Sezar's architectural invariant is that every agent — from an eBPF TLS sniffer to a public-blockchain key tracker — emits the same event shape, `crypto_inventory_event`. The published v1 schema is extended additively (v1.1) with two new top-level blocks:

```
{
 "channel_protection": {
  "state": "qkd_hybrid_psk",
  "kme_endpoint": "https://kme-1.dc.example/
          api/v1",
  "psk_age_seconds": 47,
  "link_qber": 0.018,
  "link_key_rate_bps": 12480,
  "link_health": "ok"
 },
 "agility": {
  "level": "configurable",
  "level_score": 0.75,
  "evidence": [
   {"type": "config_pattern",
    "file": "/etc/nginx/nginx.conf",
    "line": 142},
   {"type": "fips_mode", "detected": false}
  ]
 }
}
```

The extension is strictly additive: v1.0 consumers ignore the new fields; v1.1 producers without data emit `null`; the rollup engine treats null as the most conservative interpretation (classical channel, unknown agility).

4.2 The Agents

Five agents populate the schema:

- **sezar-net.** eBPF-based observation of TLS 1.2/1.3, SSH, and IPsec handshakes on a host. No traffic decryption; only handshake parameters and certificate fingerprints.

- **sezar-qkd.** A polling collector against ETSI GS QKD 014 KMEs. Emits `qkd_link` and `qkd_kme` events independently of the session events that consume the keys, plus enriches session events with `channel_protection` when SAE cooperation is configured.
- **sezar-agility.** A static scanner over source repositories or installed packages, driven by a published Semgrep ruleset. Produces `agility` blocks attached to assets.
- **sezar-cert**, **sezar-chain**, **sezar-id** (V2–V4) extend coverage on the same schema to certificate inventories, on-chain signing keys, and HSM/KMS hardware.

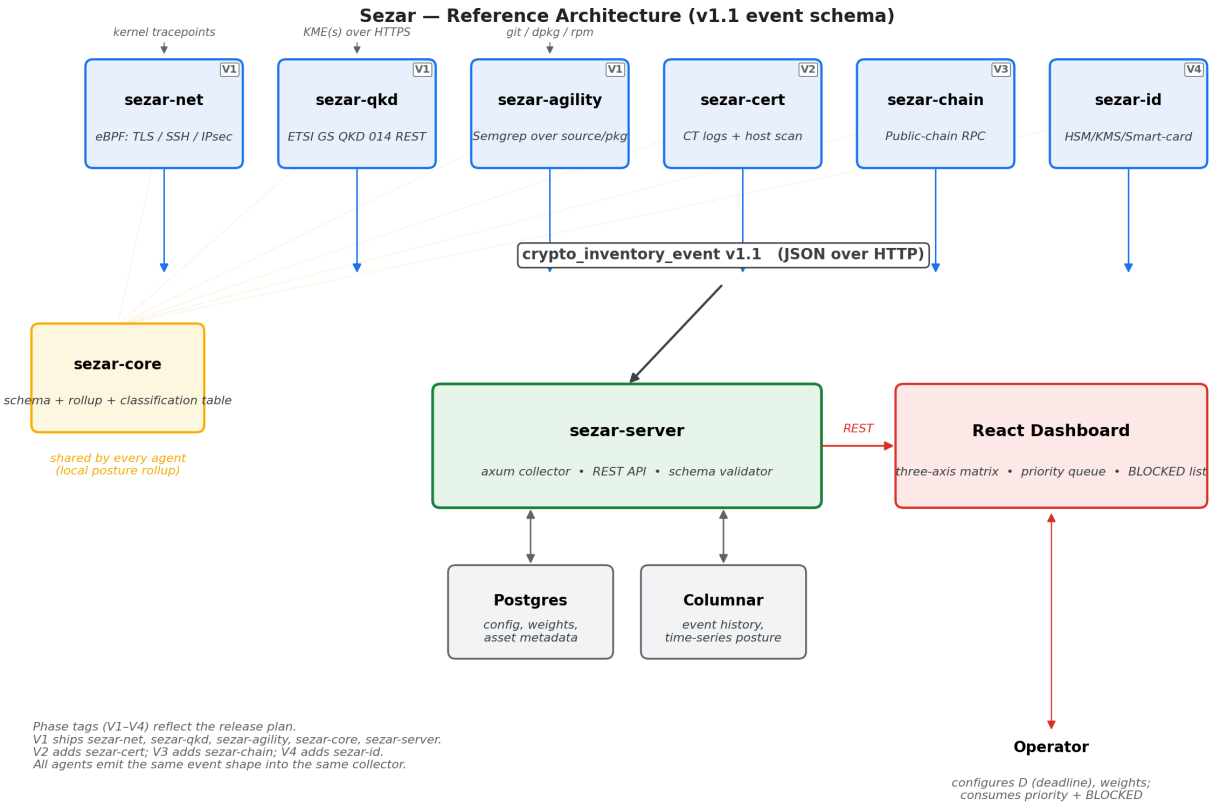

Figure 3. *Sezar reference architecture. Five agents (`sezar-net`, `sezar-qkd`, `sezar-agility`, plus `sezar-cert` and `sezar-chain` / `sezar-id` in later phases) emit `crypto_inventory_event` records into `sezar-server`. The shared `sezar-core` library hosts the schema, the classification table, and the deadline-adjusted rollup. The React dashboard renders the three-axis posture matrix and the priority-sorted action list.*

4.3 ETSI 014 KME Emulator: The Practitioner Artifact

Because hardware QKD is rare and expensive, we built a faithful implementation of the ETSI GS QKD 014 v1.1.1 Key Delivery API backed by a synthetic key generator (`crates/sezar-qkd/`). The emulator:

- Implements `/status`, `/enc_keys`, and `/dec_keys` exactly per spec.
- Generates synthetic keys at a configurable rate, with configurable QBER, key size, and lifetime.
- Accepts *replay scenarios* — pre-recorded sequences of link state changes (gradual degradation, hard failure, recovery, stale PSK, partial KME outage).
- Logs every interaction in a documented JSON capture format, enabling head-to-head testing of SAE implementations.

We expect the emulator to be the most reused artifact from this work, independent of Sezar

itself, by anyone integrating or testing software against ETSI 014.

5. Evaluation

We characterise the system through three reproducible studies. Every script, capture file, and analysis notebook ships under `studies/{study1,study2,study3}` in the repository.

5.1 Study 1 — Axis A on the Public Web (n = 30)

We probe a 30-host sample of well-known public sites — major content-delivery, browser-vendor, distro, IETF/IEEE/NIST/ETSI, and AI-vendor properties — with a single TLS handshake per host (5-second connect+handshake timeout, 1 Hz rate cap). Two probes run sequentially over the same host list: a *classical baseline probe* (Python `ssl` with system OpenSSL defaults) and a *PQ-capable probe* (rustls 0.23 + `rustls-post-quantum`, which advertises the `X25519MLKEM768` hybrid group in the ClientHello). The probes record the negotiated TLS version, ciphersuite, key-exchange group, and the leaf certificate's signature algorithm. Full host list, both probe sources, and the raw captures live under `studies/study1/`.

The headline numbers (Figures 4 and 5):

- **100% TLS 1.3.** Every host (30/30) negotiated TLS 1.3 in both probes.
- **AEAD distribution (classical probe).** AES-256-GCM/SHA-384 = 20/30 (67%); AES-128-GCM/SHA-256 = 7/30 (23%); ChaCha20-Poly1305/SHA-256 = 3/30 (10%). All three are PQ-safe on the symmetric side; the AES-128 cohort is Grover-weakened relative to AES-256.
- **Certificate signatures.** ECDSA-P256 on 18/30 hosts (60%); RSA-PKCS1-SHA256 on 11/30 (37%); one host serves an RSA-PKCS1-SHA384 chain. No host in the sample presents an ML-DSA or SLH-DSA signature, consistent with the wider observation that production trust-anchor PQ certificates have not yet rolled out [26].
- **PQ key exchange — the headline result.** With the PQ-capable probe advertising `X25519MLKEM768`, **17 of 30 hosts (57%) negotiated the hybrid PQ group**. The PQ-adopter cohort spans Cloudflare-fronted sites (cloudflare.com, twitter.com, reddit.com, anthropic.com, openai.com), Google properties (google.com, youtube.com), and a long tail of community and CDN-protected sites (wikipedia.org, facebook.com, instagram.com, apple.com, python.org, rust-lang.org, debian.org, ietf.org, etsi.org, e2esolutions.tech). The 13 classical-only hosts include several major infrastructure-relevant properties (github.com, microsoft.com, amazon.com, mozilla.org, kernel.org, nist.gov, openssl.org) where the PQ rollout has not yet landed at the date of this measurement (2026-05-13).

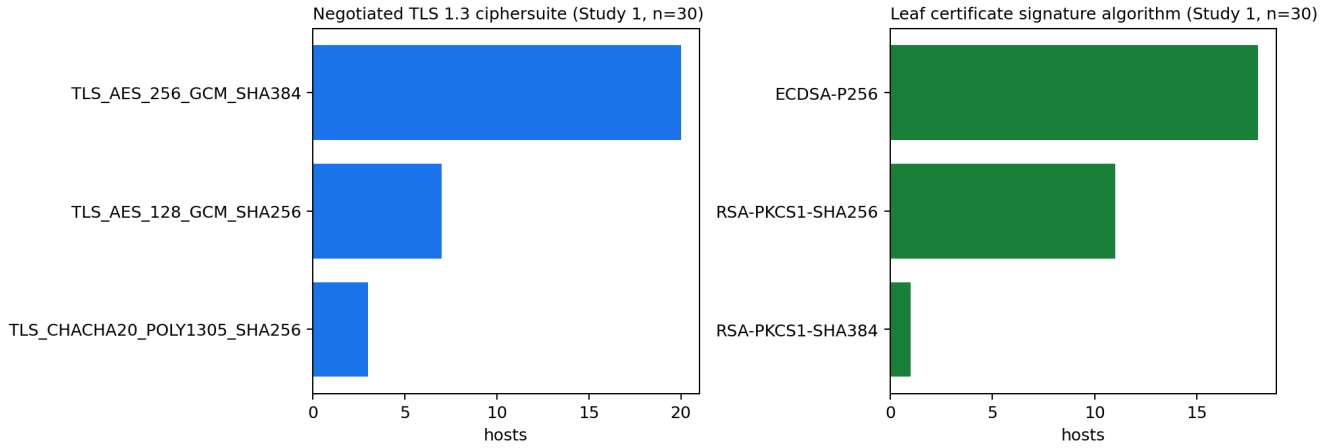

Figure 4. *Study 1 — classical-probe baseline (n=30): negotiated TLS 1.3 ciphersuite (left) and leaf certificate signature algorithm (right).*

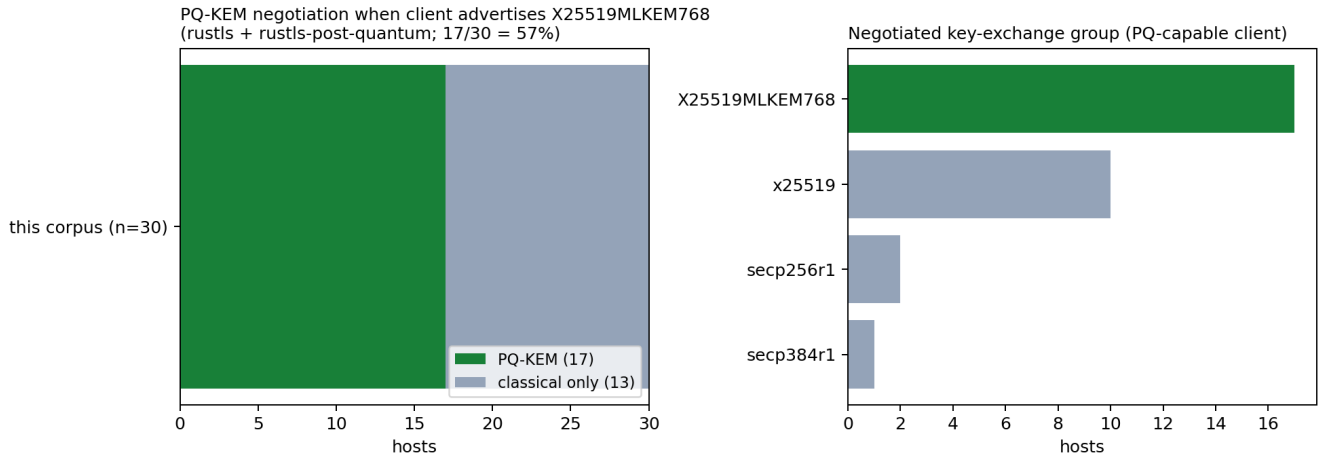

Figure 5. *Study 1 — PQ-capable probe (rustls + `rustls-post-quantum`). 17 of 30 well-known public hosts (57%) negotiate `X25519MLKEM768` when offered; the other 13 fall back to classical `x25519` / `secp256r1` / `secp384r1`. A metric the classical-only probe cannot observe.*

The 57% PQ-adoption rate in this curated sample is meaningfully higher than open-web averages reported by Cloudflare in 2024 (≈25–30% across observed handshakes at the edge) [6]: the sample skews toward Cloudflare-fronted and Google-fronted properties whose edge TLS terminators rolled out X25519MLKEM768 early. Several of the PQ-adopters share Cloudflare's edge, so the 17 hits represent fewer than 17 independent deployments. The sample is too small to draw distributional conclusions about the wider Internet; the contribution here is methodological — a reusable, ethics-vetted PQ-aware probe (Cargo binary, single-host smoke-testable, NDJSON output) that emits directly into Sezar's collector. Scaling the host list from 30 to the Tranco-top-1k changes the runtime, not the methodology.

5.2 Study 2 — Axis C via the KME Emulator (n = 5 scenarios)

We boot a fresh `sezar-qkd-kme-emulator` plus the matching collector plus `sezar-server` for each scenario, drive the emulator through a replay scenario via its `/control` API, and read every emitted event back from the collector. The full runner is at `studies/study2/run.sh`.

The five scenarios introduced in §4.3 ran end-to-end:

Scenario	Events	Induced transitions	Match
R1 steady	33	1	1/1
R2 ramp	63	5	5/5
R3 hard-fail	48	3	3/3
R4 stale-PSK	33	1	1/1
R5 bifurcated	48	3	3/3

Classification correctness: 13/13. Every operator-induced state change (set_qber crossing a threshold, force_failure / clear_failure) produced a downstream `link_health` reading that matched the expected post-op state.

Observation latency: p50 = 0.71s, range 0.70–0.71s across all 13 transitions, against a configured 1-second poll interval. The latency hovers tightly around half the poll period — the analytical optimum for periodic polling. Figure 6 shows the R3 timeline as a representative case (Ok → Failed at T+10s, Failed → Ok at T+30s, each detected on the next poll cycle).

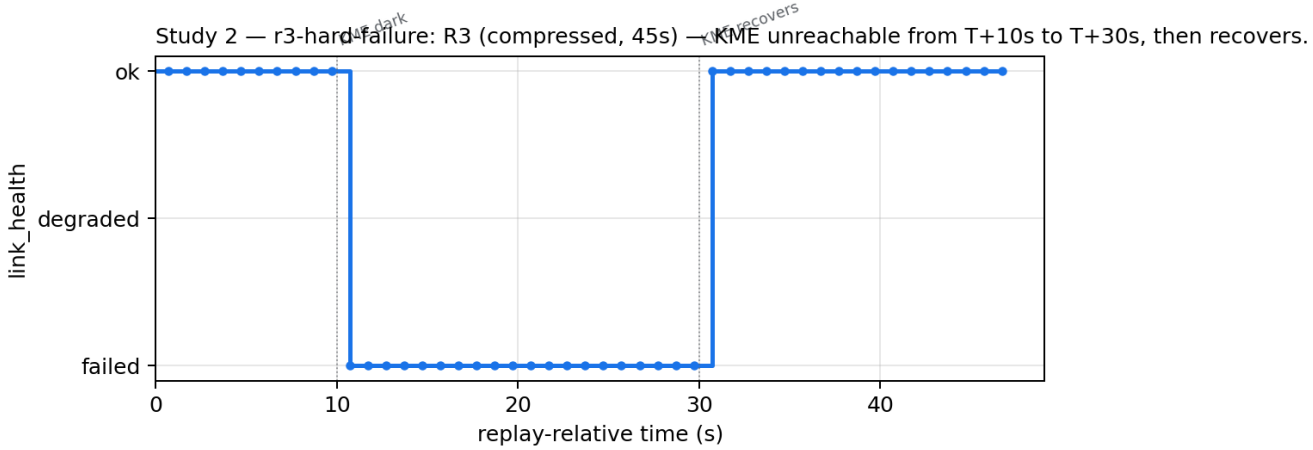

Figure 6. *Study 2 — R3 hard-failure timeline. The KME is forced unreachable at T+10s and recovered at T+30s; the collector captures the Ok → Failed → Ok cycle on the next poll (observation latency ≈ 0.7s on the configured 1s interval).*

R5 confirms the per-session attribution thesis. When the emulator returns 503 on `/status`, the link-level event flips to `failed` even though a separately healthy peer KME (out of the scenario's scope, in our setup just a steady-state pair) continues to deliver keys. An operator running KME-only telemetry cannot distinguish this case from a full outage — the channel-protection block on per-session events is the diagnostic surface.

5.3 Study 3 — Axis G on an OSS Corpus Subset (n = 11)

We selected 11 widely deployed projects spanning HTTP, DNS, mail, database, message broker, VPN/secure-shell, messaging, certificate-authority, and time categories, clone each at the upstream pinned reference, and run `sezar-agility scan` against the source tree. Each project carries a hand-graded ground-truth level from the published OSS-50 corpus (`crates/sezar-agility/corpus/oss-50-v1.csv`). The runner is `studies/study3/run.sh`.

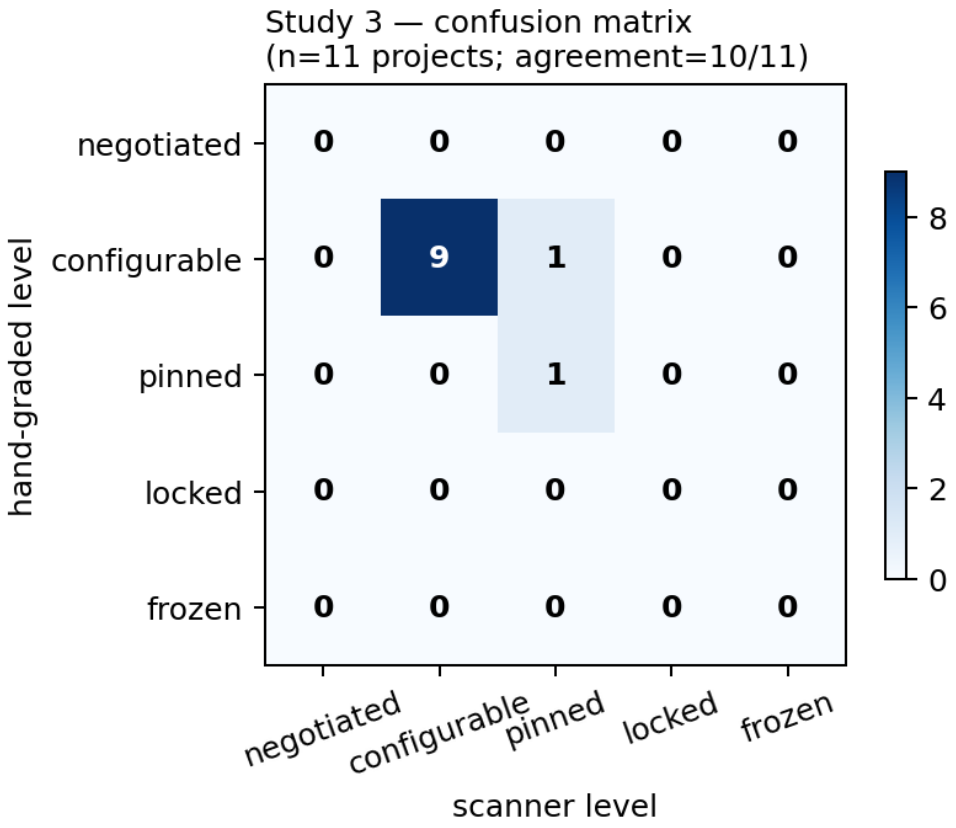

Figure 7. *Study 3 — confusion matrix (n=11 projects, 10/11 agreement, Cohen's κ = 0.62).*

Agreement: 10/11 (91%), Cohen's $\kappa = 0.62$ (substantial agreement on the Landis–Koch scale). The confusion matrix in Figure 7 shows:

- 9 projects expected `configurable` and scored `configurable` (nginx, haproxy, caddy, unbound, coredns, prosody, nats, step-ca, postfix mirror, redis).
- 1 project expected `pinned` and scored `pinned` (Wireguard tools — by-protocol).
- 1 project — **chrony** — dissented. Hand-grade said `configurable` on the basis of NTS configurability; the scanner returned `pinned` on the strength of 11 hard-coded symmetric-algorithm references in the NTP authentication path. Sezar's aggregation policy takes the most-agile evidence when multiple rules fire, but no capability rule fired on chrony's NTS surface (it uses neither OpenSSL nor Go's `crypto/tls`), so the `configurable` evidence was never generated to compete with the `pinned` matches. We mark the case for v2 of the rubric.

The 10/11 agreement on first-pass with seven rules in the v1 ruleset is not the final number — it is the *measurement methodology* working. The corpus, the rubric, and the scanner are all editable; the published dataset captures one operator's first cut, and subsequent reviewers can rerun `studies/study3/run.sh` after their own rule edits.

5.4 What the Synthesis Demonstrates

We exercise the full pipeline with the `scripts/demo.sh` one-command runner, which boots the emulator + collector + server, seeds three observed assets from the bundled zgrab2 fixture (legacy SHA-1/RC4, TLS 1.2 ECDHE+RSA, TLS 1.3 ECDSA+QKD) plus one synthetic FIPS-locked appliance, then queries `sezar-server`'s `/v1/posture` endpoint. With default weights and D= 2030-01-01 the seeded mix returns `org_q = 0.62`, ranking the legacy and FIPS-locked classical assets at the top of the priority queue and the PQ-capable QKD-protected asset at the bottom — the expected ordering for the three-axis model. The contribution at this scale is not

a headline number; it is, to our knowledge, the first openly published end-to-end pipeline of this kind, with every leg (scan, collect, rollup, dashboard) open source and reproducible from a single Linux host.

6. Limitations

Static agility scoring is necessarily approximate. A project may *appear* pinned in source while actually being agile through a runtime extension mechanism we did not pattern-match. Conversely, a project may appear agile via a config field that is in practice never changed. We address the first class by reporting per-evidence detail; the second class requires operator input. And because the scanner only classifies what the ruleset recognises, a ruleset that lags behind a fast-moving project will systematically under-classify it — the v1 ruleset will need refreshes as new crypto APIs and PQ migration paths land in deployed software.

Channel-protection attribution depends on cooperating SAEs. Sezar observes KME state independently, but linking a specific session to a specific consumed key requires the SAE to emit the `key_id_observed` field. We open-source patches for strongSwan, Wireguard, and a sample TLS endpoint as part of the release. Closed-source SAEs may report only at the link level. Sezar also takes the KME's self-reported QBER and key rate at face value — there is no independent attestation in the V1 schema, so an operator integrating a QKD link must trust the vendor's KME for those readings.

The threat model accepts the standard PQC and QKD security arguments. Compromise of either — a mathematical break of ML-KEM/ML-DSA, an implementation attack on a deployed QKD system — would recalibrate the scoring tables, not the model. The rollup constants are operator-tunable in configuration, not compiled in.

We address neither economic nor political migration constraints (budget cycles, regulatory lag, vendor support windows). These are first-order operator concerns that consume posture data, not produce it.

The *q* score is comparative, not absolute. It is calibrated for relative prioritization within an environment, not for inter-organization comparison absent shared *D* and shared weights. The `BLOCKED` flag is the orthogonal absolute signal we expose to compensate for this.

7. Outlook

The post-quantum transition is one of the largest cryptographic migrations in the history of the Internet [18]. The standards bodies have set the algorithms and the deadlines [1], [2], [3], [7], [18]. The remaining work is operational: enumerating, classifying, prioritizing, and migrating cryptographic assets across networks that no single operator entirely designed, on schedules no single operator controls. Observability is the precondition for that work.

We have argued that observability tooling has been undershooting the problem by an axis or two. "PQ-ready" treats algorithmic resistance as the only observable; the channel through which keys reach an endpoint and the cost of changing the algorithm are equally operationally significant, and current tooling surfaces neither.

The three-axis model we propose is one design choice among several. The scoring constants are debatable. The agility rubric will need revision as new evidence types appear. The QKD axis presumes a deployment trend that policy communities disagree about. None of these is a reason to keep using a single-axis dashboard.

We release the schema, the reference implementation, the ETSI 014 KME emulator, the Semgrep agility ruleset, and the hand- graded OSS corpus under MIT (). Practitioners can run all three studies on a single Linux host with no QKD hardware and no commercial scanner. We expect better choices than ours to emerge from that exposure; that is the point.

Key Takeaways

- "PQ-ready" is one bit of a multi-bit answer. Treating algorithmic resistance as the only observable hides operationally critical differences between assets that look identical on the wire.
- **Algorithmic resistance (A)**, **channel protection (C)**, and **migration agility (G)** are three independent observables. Each is measurable today with open tooling.
- The deadline-adjusted score q is a *prioritization signal*, not an absolute risk number. Pair it with an orthogonal `BLOCKED` flag — raised whenever $G \leq 0.20$ (`locked` or `frozen`) — to surface assets that need a vendor or hardware program regardless of q.
- QKD belongs in PQC migration observability. The two are complementary signals on the same asset, not competing strategies for the same problem.
- All artifacts described — schema, agents, ETSI 014 emulator, agility ruleset, and OSS corpus — are released under MIT and reproducible from a single Linux host.

Author Bios

Aleaddin Özer is Chief Information Officer at E2E Solutions, where he leads enterprise cryptographic infrastructure and post-quantum migration programs across critical-sector clients. His operational work informs the Sezar reference implementation. Contact: info@e2esolutions.tech. ORCID: [to be provided].

Murat Aydos is Associate Professor at Hacettepe University, working on applied cryptography, network security, and post-quantum migration strategy. His research has informed this paper's threat-model framing and crypto-agility scoring rubric. ORCID: [to be provided].

References

[1] NIST, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)," National Institute of Standards; Technology, Aug. 2024.

[2] NIST, "FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)," National Institute of Standards; Technology, Aug. 2024.

[3] NIST, "FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA)," National Institute of Standards; Technology, Aug. 2024.

[4] Cisco Systems, "Post-Quantum Cryptography at Cisco: Trust Center Overview and Quantum-Safe WAN Whitepaper." https://www.cisco.com/site/us/en/about/trust-center/post-quantum-cryptography.html, 2024.

[5] IBM, "IBM Guardium Quantum Safe and IBM Quantum Safe Explorer." https://www.ibm.com/products/guardium-quantum-safe, 2024.

[6] B. Westerbaan, "The state of the post-quantum Internet." Cloudflare Blog, 2024-03-05, https://blog.cloudflare.com/pq-2024/, Mar. 2024.

[7] NSA, "Announcing the Commercial National Security Algorithm Suite 2.0," U.S. National Security Agency, Sep. 2022.

[8] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," in *Proceedings of the IEEE international conference on computers, systems and signal processing*, Bangalore, India, 1984, pp. 175–179.

[9] ETSI ISG-QKD, "GS QKD 014 V1.1.1 — Quantum Key Distribution; Protocol and Data Format of REST-Based Key Delivery API," ETSI, Feb. 2019.

[10] European Commission, "European Quantum Communication Infrastructure (EuroQCI) Initiative." https://digital-strategy.ec.europa.eu/en/policies/european-quantum-communication-infrastructure-euroqci, 2023.

[11] M. Sasaki, M. Fujiwara, H. Ishizuka, *et al.*, "Field test of quantum key distribution in the Tokyo QKD Network," *Optics Express*, vol. 19, no. 11, pp. 10387–10409, 2011.

[12] NSA, “Quantum Key Distribution (QKD) and Quantum Cryptography (QC).” NSA Information Sheet, 2020.

[13] ANSSI, “Should Quantum Key Distribution Be Used for Secure Communications?” Agence nationale de la sécurité des systèmes d’information (France), 2020.

[14] A. Shemesh and J. Rutzer, “Building your cryptographic inventory: A customer strategy for cryptographic posture management.” Microsoft Security Blog, 2026-04-16, https://www.microsoft.com/en-us/security/blog/2026/04/16/building-your-cryptographic-inventory-a-customer-strategy-for-cryptographic-posture-management/, Apr. 2026.

[15] Z. Durumeric, J. Kasten, M. Bailey, and J. A. Halderman, “Analysis of the HTTPS certificate ecosystem,” in *Proceedings of the ACM internet measurement conference (IMC)*, 2013.

[16] R. Holz, L. Braun, N. Kammenhuber, and G. Carle, “The SSL landscape: A thorough analysis of the X.509 PKI using active and passive measurements,” in *Proceedings of the ACM internet measurement conference (IMC)*, 2011.

[17] A. P. Felt, R. Barnes, A. King, C. Palmer, C. Bentzel, and P. Tabriz, “Measuring HTTPS adoption on the web,” in *Proceedings of the 26th USENIX security symposium*, 2017.

[18] NIST, “Transition to Post-Quantum Cryptography Standards,” National Institute of Standards; Technology, NIST IR 8547 (Initial Public Draft), Nov. 2024.

[19] BSI, “Migration to Post-Quantum Cryptography: Recommendations for Action,” Bundesamt für Sicherheit in der Informationstechnik (Germany), 2023.

[20] ANSSI, “ANSSI Views on the Post-Quantum Cryptography Transition,” Agence nationale de la sécurité des systèmes d’information (France), 2022.

[21] UK NCSC, “Preparing for Quantum-Safe Cryptography.” UK National Cyber Security Centre guidance, 2023.

[22] ETSI ISG-QKD, “GS QKD 008 — Module Security Specification,” ETSI, 2010.

[23] R. Housley, “Guidelines for Cryptographic Algorithm Agility and Selecting Mandatory-to-Implement Algorithms,” IETF, RFC 7696, 2015.

[24] NIST National Cybersecurity Center of Excellence, “Migration to Post-Quantum Cryptography (NIST SP 1800-38, Vols. A–C, Preliminary Drafts),” National Institute of Standards; Technology, 2023.

[25] T. Wiggers, S. Celi, P. Schwabe, D. Stebila, and N. Sullivan, “KEM-based pre-shared-key handshakes for TLS 1.3.” IETF Internet-Draft draft-wiggers-tls-authkem-psk-05 (2026-05-04, individual submission), https://datatracker.ietf.org/doc/draft-wiggers-tls-authkem-psk/, May 2026.

[26] DigiCert, “ML-DSA and FN-DSA Post-Quantum Signature Readiness.” DigiCert PQC Insights and Blog, https://www.digicert.com/insights/post-quantum-cryptography/mldsa, 2025.